

TP Broadcasting : Laravel Reverb

Dans ce TP, on va créer un projet de Chat Temps Réel.

1 : Création du projet

```
composer create-project Laravel/Laravel 021-broadcasting-chat-reverb
cd 021-broadcasting-chat-reverb
php artisan install:broadcasting
```

Lancer les serveurs :

```
php artisan serve
php artisan reverb:start
```

2 : Configuration

Dans `.env` :

```
BROADCAST_CONNECTION=reverb
QUEUE_CONNECTION=sync
```

3 : Modèle Message

```
php artisan make:model Message -m
```

Migration :

```
Schema::create('messages', function (Blueprint $table) {
    $table->id();
    $table->text('content');
    $table->timestamps();
});
```

```
php artisan migrate
```

4 : Event Broadcasting

```
php artisan make:event MessageSent
```

Code :

```
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;

class MessageSent implements ShouldBroadcast
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $message;
    public function __construct($message)
    {
        $this->message = $message;
    }
}
```

```

    }

    public function broadcastOn(): array
    {
        return ['chat'];
    }
}

```

5 : Contrôleur

```
php artisan make:controller ChatController
```

```

use App\Events\MessageSent;
use App\Models\Message;
use Illuminate\Http\Request;

class ChatController extends Controller
{
    public function index()
    {
        $messages = Message::latest()->take(30)->get()->reverse();
        return view('chat', compact('messages'));
    }

    public function send(Request $request)
    {
        $message = Message::create([
            'content' => $request->input('content'),
        ]);

        // Diffusez le message à tous les clients connectés
        event(new MessageSent($message->content));

        return response()->json(['status' => 'Message sent!']);
    }
}

```

NB: Ajouter le champ « content » dans la propriété \$fillable du modèle Message ;

```
protected $fillable = ['content'];
```

6: Routes

```

use App\Http\Controllers\ChatController;

Route::get('/', [ChatController::class, 'index']);
Route::post('/send', [ChatController::class, 'send']);

```

7 : Vue Blade

```
resources/views/chat.blade.php
```

```
<!DOCTYPE html>
```

```

<html>

<head>
  <title>Chat</title>
  <meta name="csrf-token" content="{{ csrf_token() }}">
  <script src="https://cdn.tailwindcss.com"></script>
  @vite(['resources/js/app.js'])
</head>

<body class="bg-gray-100 p-4">

  <h1 class="text-2xl font-bold text-center mb-4">Chat Temps Réel</h1>

  <div id="messages" class="bg-white p-4 rounded shadow-md mb-4">
    @foreach ($messages as $msg)
      <p class="text-gray-700">{{ $msg->content }}</p>
    @endforeach
  </div>

  <div class="flex items-center space-x-2">
    <input type="text" id="message" placeholder="Votre message"
      class="border border-gray-300 rounded p-2 flex-1">
    <button onclick="sendMessage()"
      class="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600">Envoyer</button>
  </div>

  <script>
    function sendMessage() {
      fetch('/send', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content
        },
        body: JSON.stringify({
          content: document.getElementById('message').value
        })
      });
      document.getElementById('message').value = '';
    }
  </script>

</body>

</html>

```

8 : Laravel Echo

resources/js/app.js:

```

import "./bootstrap";

window.Echo.channel("chat").listen("MessageSent", (e) => {
  let div = document.getElementById("messages");
  div.innerHTML += `<p>${e.message}</p>`;
});

```

9 : Test

Ouvrir 2 navigateurs :

Envoyer un message

Voir apparaître instantanément dans l'autre fenêtre

